UNIVERSITY VIVA GUIDE / P-CAP

Parallel Data Center Thermal Simulation and Hotspot Analysis

In short: A small digital twin: one computer model of a data center, shown as a grid of racks.

We can change one rack or select many racks and change their temperature, workload, and cooling together. The model updates the heat in each rack and shows how heat moves to nearby racks. We can watch this happen on a live heatmap. The same calculation runs on one CPU core, on multiple CPU cores with OpenMP, or on an NVIDIA GPU with CUDA. A second page compares their speed.

Team members

Soham Bhattacherjee
Saarthak Patwal
Akshansh Goel

What to say first: The project studies how heat, workload, and cooling interact across nearby racks. It also shows when parallel computing makes the same simulation faster.

This is a simple teaching model. One step is one update, not one real-world second.

Tools and architecture

In short: The browser displays results; Flask passes requests to one compiled C++ library; that library chooses CPU, OpenMP, or CUDA.

Component	Technology	Why we used it	Its job here
Simulator	C++	Fast loops over many racks	Updates temperatures and statistics
CPU parallel mode	OpenMP	Simple way to split loops	Runs rack updates on CPU workers
GPU mode	CUDA	Many GPU threads	Runs one GPU thread per rack
Build	CMake	Builds the same library for Windows	Finds OpenMP and optional CUDA
Server	Python + Flask	Small web server	Receives requests and returns JSON data
Bridge	ctypes	Calls a compiled C library from Python	Loads thermal.dll and passes arrays
Interface	HTML, CSS, JavaScript	Simple two-page interface	Controls, layout, requests, selection
Heatmap	Canvas	Draws many colored cells	Shows temperatures and outlines
Charts	Chart.js	Draws line and bar graphs	History and benchmark plots

Request path

```
Browser  ->  Flask  ->  ctypes  ->  thermal.dll
                                      |
                          CPU / OpenMP / CUDA
```

JSON is the text format used between browser and server.

Library: thermal.dll is the compiled C++ program on Windows. A plain C interface in api.cpp lets ctypes call it. CUDA is included only when CMake finds its compiler.

One grid, many racks

In short: Each square is a server rack. Each step changes its workload, then computes a new temperature using its old temperature and nearby racks.

An N x N grid has N rows and N columns. Each rack stores **temperature** (degrees C), **workload** (0-100%), and **cooling** (0-100%). The arrays are flat: `index = row * N + column`. The live page offers N = 64, 128, 256, or 512.

Which neighbors affect a rack?

-	N	-
W	C	E
-	S	-

For center rack C, the code averages North, South, West, and East. At an edge it uses only existing neighbors. A corner therefore has two neighbors; no heat wraps around the grid.

Why two temperature arrays?

```
OLD temperatures  ->  calculate every NEW temperature
      read only                write once per rack
After all racks finish: swap OLD and NEW arrays.
```

This is **double buffering**: an old grid and a new grid. In the 3 x 3 example, C reads the old values of N, S, W, and E. If C changed the old grid immediately, its later neighbors might read a mixture of old and new values. Separate arrays keep every rack on the same step.

Order within one step

1	2	3	4
Change workload by -2 to +2	Update temperature	Apply throughput throttling	Compute max, average, hotspots, throughput

The grid starts at 25 C. Cooling starts at 50%. Workload starts at a reproducible value from 45% to 55% for each rack. A **hotspot** is a rack above the chosen threshold, initially 85 C.

Formulas: the update

In short: A repeatable small workload change feeds the heat equation. Nearby temperatures, workload, cooling, and ambient temperature determine the new value.

1. Dynamic workload

```
delta = cell_hash(index, step) % 5 - 2
w_new = clamp(w_old + delta, 0, 100)
```

Symbols: index is the rack's flat position; step counts updates; w is workload in percent. The hash makes a repeatable integer delta from -2 to +2. **Meaning:** each rack's work changes a little each step. **Example:** if w_old = 60 and hash % 5 = 4, delta = 2 and w_new = 62%.

2. Neighbor average and ambient cooling

```
avg = sum(existing N, S, W, E temperatures) / count
effective_cooling = cooling + (25.0 - ambient)
```

Symbols: count is 2, 3, or 4 for corner, edge, or interior racks; ambient defaults to 25 C and its slider allows 10-45 C. Cooling initially is 50%. **Meaning:** a warmer ambient makes cooling less effective. **Example:** neighbors 70, 70, 70, 70 give avg = 70 C. Cooling 20 and ambient 30 give effective_cooling = 20 + (25 - 30) = 15.

3. Temperature after one step

```
T_new = T + alpha * (avg - T) + beta * w_new
          - gamma * effective_cooling
alpha = 0.2; beta = 0.01; gamma = 0.01
```

Symbols: T is the rack's old temperature, avg is the neighbor average, and w_new is the updated workload. alpha controls heat exchange with neighbors; beta adds workload heat; gamma removes heat through cooling. These constants are fixed in common.h. **Meaning:** heat moves toward the neighbor average, workload adds heat, and cooling removes heat.

```
T=80, avg=70, w_new=60, cooling=20, ambient=25
effective_cooling = 20 + (25 - 25) = 20
T_new = 80 + 0.2*(70-80) + 0.01*60 - 0.01*20
      = 80 - 2 + 0.6 - 0.2 = 78.4 C
```

Worked example: a cooling rack with four neighbors at 70 C.

At ambient 30 C, effective_cooling becomes 15, so the same example ends at 78.45 C. The code does not clamp temperature after this calculation.

Formulas: what we measure

In short: The updated temperature decides whether a rack throttles and whether it is counted as a hotspot. Statistics combine all racks.

4. Throttling and throughput

```
effective_workload = (T_new > 85.0) ? w_new * 0.7 : w_new
throughput = sum(effective_workload of all racks)
```

Symbols: w_new is the stored workload after drift. The throttle cutoff is fixed at 85 C. **Meaning:** a hot rack contributes only 70% of its workload to throughput; its stored workload and heat input are unchanged. **Example:** at 86 C and workload 60%, effective workload is 42; at 85 C it is 60. Together they contribute 102 units.

5. Hotspots and temperature statistics

```
hotspot = (T_new > threshold)
hotspot_count = sum(hotspot over all racks)
maximum = max(T_new over all racks)
average = sum(T_new over all racks) / (N * N)
```

Symbols: threshold starts at 85 C and the UI allows 30-120 C. N is the grid width. **Meaning:** threshold changes only the hotspot display, not the fixed 85 C throttle rule. **Example:** temperatures 78.4 and 86 give maximum 86, average 82.2, and one hotspot when threshold = 85; at threshold = 90 there are none.

6. Comparing speed

```
time_per_step_ms = measured_wall_time_ms / steps
speedup = CPU_time_per_step_ms / backend_time_per_step_ms
verified if abs(final_max - CPU_final_max) <= 0.001 C
```

Meaning: smaller time is faster; speedup above 1 means faster than sequential CPU. **Example:** CPU 10 ms and CUDA 2 ms give 5x speedup. The check compares final maximum temperature only, within 0.001 C. The benchmark uses five warm-up steps outside its timing.

How temperature becomes dynamic

Three inputs can change it: the per-rack workload drift each step; the cooling and ambient values; and user edits to one or many racks. The next grid then spreads a changed rack's heat to its neighbors over later steps.

Three ways to compute

In short: All three modes use the same rack equation. They differ in how work is assigned and where arrays live.

Sequential CPU

cpu_sim.cpp visits each row and column in ordinary nested loops. It writes one new rack value at a time. After swapping temperature arrays, one loop collects the statistics. This is the reference mode.

OpenMP: several CPU workers

omp_sim.cpp adds `#pragma omp parallel for collapse(2) schedule(static)`. **collapse(2)** combines the row and column loops into one list of rack jobs. A **thread** is one worker. Each thread writes different rack positions. The loop's end is a **barrier**: everyone finishes before the arrays swap.

The statistics loop uses `reduction(+:sum,throughput,hotspots)` and `reduction(max:maximum)`. A **reduction** combines each worker's partial answer into one result without several workers editing one counter at once.

CUDA: many GPU threads

cuda_sim.cu maps a 16 x 16 **block** (a group of 256 GPU threads) onto a 16 x 16 rack region. One thread updates one rack. The two temperature arrays, workload, and cooling stay in GPU **global memory** between steps. A selectable tiled kernel first caches an 18 x 18 region in faster **shared memory**, including a one-rack border called a halo.

A first reduction kernel combines up to 256 racks per block; a second combines the block results for max temperature, temperature sum, hotspot count, and throughput. `cudaDeviceSynchronize()` waits between steps. CUDA events time the GPU work. Full arrays return to the CPU only when the browser asks for a frame.

Mode	Runs on	Workers / memory	Useful when
CPU	CPU	One loop; host RAM	A simple reference or small grid
OpenMP	CPU cores only	Several CPU threads; shared host RAM	Enough racks to share across cores
CUDA	RTX 4060 GPU	16 x 16 thread blocks; GPU memory	Large grids with many independent racks

Key ideas: Data parallelism means using the same equation on different racks. Double buffering avoids a **race**, where results depend on execution order. Moving data between host RAM and GPU memory costs time, so CUDA keeps arrays on the GPU. Small grids may lose time to thread setup or kernel launches.

What the user can do

In short: The live page changes racks and shows the result. The benchmark page compares speed and checks the final maximum temperature.

Page 1: Live Simulation

Choose **CPU, OpenMP, or CUDA** and a **grid size**; either choice resets the grid. **Start** runs repeatedly, **Pause** stops, **Step +1** runs once, and **Reset** restores initial data. The speed slider chooses steps per requested frame, not guaranteed frames per second.

Click to select one cell; drag to select a rectangle; Shift-click to add or remove one cell. Select all and Clear selection work on the whole grid. Apply sets temperature, workload, and cooling for every selected rack. Quick buttons add 10 C, subtract 10 C, set max workload, or set workload to zero. The last action is temporary: workload drift resumes next step.

Presets are Normal load, Hot aisle failure, Cooling failure, Cooling boost, and Random spikes. The heatmap shows rack color, white selection outlines, and yellow hotspot outlines. Hover shows a rack's three values. Six cards show max/average temperature, hotspots, throughput, step, and ms per step. A line chart tracks max temperature and throughput.

Page 2: Performance Comparison

Run Benchmark tests 128, 256, 512, 1024, and 2048-wide grids. The **bar chart** uses grid size on X and wall milliseconds per step on Y; shorter bars are faster. The **line chart** uses grid size on X and speedup over CPU on Y; higher is faster. The table lists times and speedups. Its check compares final maximum temperature with CPU within 0.001 C. The page also shows OpenMP threads, GPU name, CUDA block size, and a CSV download.

Settings: adjustable versus fixed

Setting	Default	Allowed / where
Live grid size	64	64, 128, 256, 512 (UI)
Steps per frame	5	1-30 (UI); /step accepts 0-100
Benchmark size / steps	128-2048 / 100	Five fixed sizes / 1-1000 steps
Ambient / hotspot threshold	25 C / 85 C	10-45 C / 30-120 C
Selected temp / work / cool	65 C / 100% / 25%	UI: 0-120 C / 0-100% / 0-100%
alpha / beta / gamma	0.2 / 0.01 / 0.01	Fixed in code; no UI control
CUDA block / tiling	16 x 16 / off	Block fixed; tiling checkbox on/off
OpenMP workers	8 for run-cuda.ps1	OMP_NUM_THREADS environment variable

Run it and explain it

In short: Start the saved CUDA build from the project folder, open the local page, then use these short answers to explain the design.

```
cd C:\Users\Saarthak\Desktop\datacenter_final\datacenter-thermal
powershell -NoProfile -ExecutionPolicy Bypass -File .\run-cuda.ps1
Open http://127.0.0.1:5000  ->  choose CUDA in the UI
```

The script sets `THERMAL_LIBRARY` to build-cuda-vs/Release/thermal.dll and `OMP_NUM_THREADS=8`, adds the OpenMP runtime path, then runs `server/app.py`. Keep the terminal open; Ctrl+C stops the server. Rebuild with `.\build-cuda.cmd` after changing C++ or CUDA source.

Ten likely viva questions

1. Why double buffering? Every rack reads the old grid. A separate new grid stops update order from changing the answer.

2. Why one GPU thread per rack? Each rack runs the same small equation and writes its own output, so the jobs are independent.

3. Why is OpenMP CPU-only here? Its parallel loops use CPU workers sharing host memory. CUDA launches work on the NVIDIA GPU.

4. What is a reduction? It combines many partial values, such as local maxima or counts, into one final statistic.

5. What makes parallel mode faster? Many rack updates happen at once. Setup and memory costs can outweigh that gain for small grids.

6. What happens above 85 C? Effective workload becomes 70% of stored workload, reducing reported throughput.

7. How do several racks change at once? The UI sends indices or a rectangle. CPU loops or a GPU edit kernel update those racks.

8. Why can the backends agree? They use the same equation, old/new arrays, and a hash based on rack index and step.

9. What is shared-memory tiling? A CUDA block caches a 16 x 16 region plus a one-cell halo before updating its racks.

10. What does the benchmark verify? It checks final maximum temperature against CPU within 0.001 C. Speedup is CPU time divided by backend time.